Week 5 Assignment: Using the Observer Pattern to Assess Parking Charges

Master of Science

Information Technology

Object-Oriented Mthd & Pgm II

Gabriel Twizerimana

University of Denver College of Professional Studies

May 3, 2026

Faculty: Nathan Braun, MS

Dean: Bobbie Kite, PhD

Table of Contents

Implementation Report: Automated Parking Management System 1
Overview 1
Design Explanation 2
Relationship Summary 2
UML Class Diagram 3
Pattern Analysis 4
Comparison to Formal Definition: 4
Challenges and Solutions 5
Conclusion 6
References 7

Implementation Report: Automated Parking Management System

Overview

The purpose of this assignment was to design and implement a robust, decoupled parking management system using Java. The primary objective was to move away from a monolithic structure where the ParkingLot handled its own administrative tasks and instead implement a behavioral design pattern to automate event handling.

In this implementation, the system tracks vehicle entries and exits, maintains a record of active stays, and automatically calculates fees upon departure. By leveraging the **Observer** and **Strategy** design patterns, the system achieves high cohesion and low coupling, ensuring that the physical logic of the parking lot is separated from the financial logic of transaction management.

Design Explanation

Relationship Summary

The ParkingLot maintains a "one-to-many" relationship with ParkingObserver. The TransactionManager is a concrete implementation of that observer. This relationship is established in the ParkingService layer, where the manager is "subscribed" to the lot. This ensures that the ParkingLot remains entirely unaware of how transactions are managed or how fees are calculated, focusing solely on the movement of vehicles.

The architecture of this system is centered on the relationship between three core components: ParkingLot, Observer, and TransactionManager.

- **ParkingLot (The Subject):** This class represents the physical infrastructure. It maintains a collection of observers. When a car enters or exits, the lot does not process the data itself; instead, it creates a ParkingEvent and broadcasts it to all registered observers.
- **ParkingObserver (The Interface):** This is a functional interface that defines the contract for any object wishing to listen to parking events. It contains a single update(ParkingEvent event) method, adhering to the dependency inversion principle.
- **TransactionManager (The Observer):** This class implements ParkingObserver. It is the "Information Expert" for financial records. It maintains a Map of active transactions and a List of completed ones.

UML Class Diagram

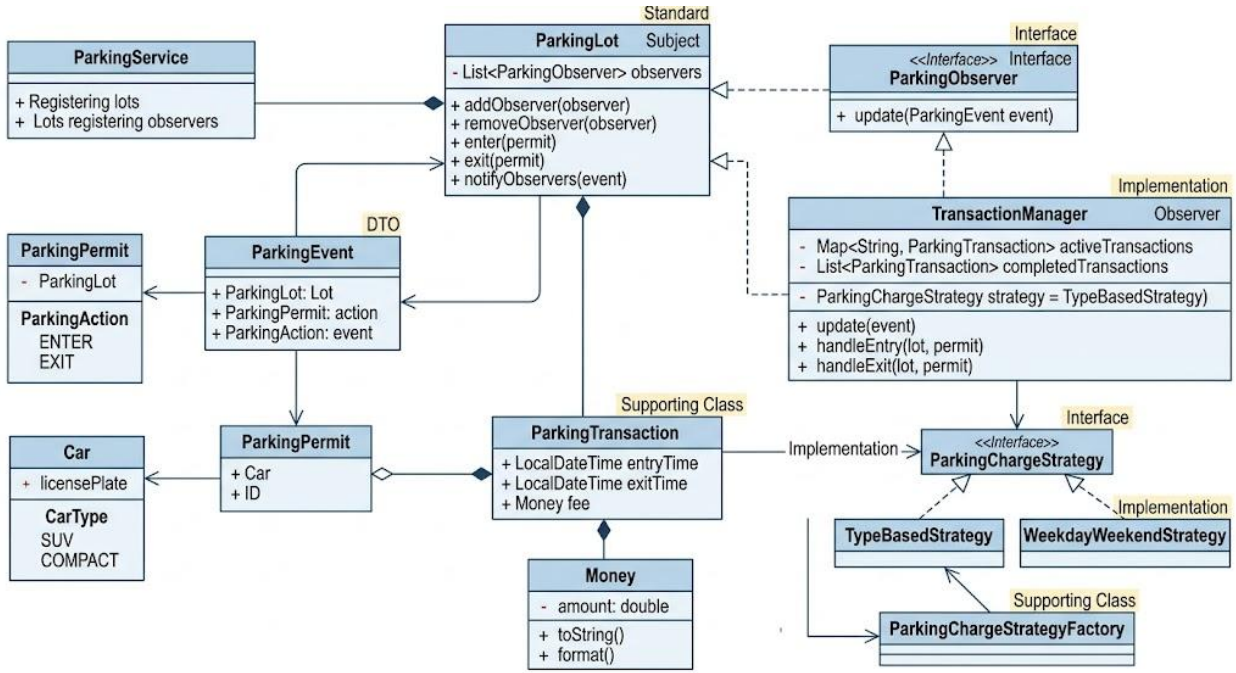

Figure 1: UML Class Diagram illustrating the Observer and Strategy design patterns

Pattern Analysis

This implementation is a textbook example of the **Observer Pattern**. According to the formal definition, the Observer pattern defines a dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

Comparison to Formal Definition:

1. **State Change:** In this system, the state change is the entry or exit of a vehicle.
2. **Notification:** The ParkingLot (Subject) iterates through its observerCollection to call the update method.
3. **Decoupling:** As defined by the textbook, the subject and observers can vary independently. In my implementation, I could add a DisplayManager observer (to update a digital "Spaces Available" sign) without changing a single line of code in the ParkingLot or TransactionManager.

Unlike a simple "push" model where only a status code is sent, my implementation uses an **Event-Object** approach. By passing a ParkingEvent object, the observer receives the full context (the Lot, the Permit, and the Action), allowing it to pull exactly the data it needs to fulfill its role.

Challenges and Solutions

One of the primary technical difficulties encountered was **distinguishing between entry and exit events** within a single update method. Initially, the update method received only a ParkingPermit. This was insufficient because the observer did not know whether to start a new transaction or finalize an existing one.

The Solution: I introduced a ParkingAction Enum (containing ENTER and EXIT) and a ParkingEvent DTO (Data Transfer Object). By wrapping the permit and the action into a single event object, the TransactionManager could use a clean switch statement or if-else logic to branch its behavior.

Another significant challenge involved **NullPointerExceptions (NPEs)** during the final receipt generation. Because the TransactionManager and ParkingLot are decoupled, the TransactionManager occasionally lacked the reference back to the ParkingLot object needed to print the lot's name on the receipt.

The Solution: I refactored the ParkingTransaction constructor to require a ParkingLot reference and ensured the ParkingEvent carried the Subject instance. This followed the **Information Expert** principle, ensuring that the transaction object itself held all the data required to describe the parking event fully, rather than relying on external lookups.

Finally, transitioning from **Mock-based testing** to **Real-Object Integration testing** revealed several logic gaps in the Money and Strategy classes. By using real objects in the ObserverTest, I discovered that the SUV multiplier was being applied incorrectly. Correcting this in the concrete Strategy classes ensured that the integration between the Observer notification and the Strategy calculation was seamless.

Conclusion

By implementing the Observer pattern, the parking system has been transformed into a modular, extensible application. The ParkingLot is now a pure representation of the domain, while the TransactionManager acts as a specialized data processor. This architecture not only meets the current requirements for automated billing but also provides a scalable foundation for future enhancements, such as loyalty programs or dynamic peak-hour pricing.

References

Gamma, Erich, Richard Helm, Ralph Johnson, and John Vlissides. 1994. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.

Larman, Craig. 2004. *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. 3rd ed. Upper Saddle River, NJ: Prentice Hall.

Oracle. 2026. "Java Documentation: The Observer Pattern and PropertyChangeEvents." Accessed May 3, 2026. https://docs.oracle.com/en/java/.